

扑克决策服务 - API 规范

版本：15.07.2026

本文档提供扑克决策服务的完整 API 规范，包括连接方式、身份认证和 JSON 消息协议。

目录

- 1. 概述
- 2. 连接方式
 - 基于 HTTP/1.1 的 WebSocket
 - 基于 HTTP/2 的 WebSocket (RFC 8441)
 - HTTP/2 CONNECT 隧道
 - HTTP 请求/响应 API
 - 协议对比
 - 连接保活
- 3. 身份认证
- 4. JSON 消息协议
 - 消息类型概览
 - gameEvents
 - getAnswer
 - fullGameLog
 - playerAction (响应)
 - 错误响应
- 5. 牌局事件类型
- 6. 游戏类型及变体
 - 标准游戏
 - BOMB 变体
 - DOUBLE BOARD BOMB 变体
 - SQUID 变体
 - RIPPER 变体
 - POFC 变体
- 7. 协议流程
- 8. 超时与限制
- 9. 完整示例

1. 概述

扑克决策服务通过基于 WebSocket 的 API 提供实时扑克决策建议。客户端发送牌局状态信息，并接收最优行动建议。

主要特性

项目	说明
传输方式	WebSocket（主要方式）、HTTP 请求/响应（备选方式）
消息格式	JSON（UTF-8 编码）
安全性	所有连接均须使用 TLS
身份认证	基于令牌，连接后的第一条消息用于认证
会话管理	使用会话 ID 进行重连和状态恢复
金额单位	筹码（1 筹码 = 最小货币单位，例如 0.01 美元）

服务端点

```
wss://api.example.com/?pid={playerId}
https://api.example.com/api/command
```

注意：请将 `api.example.com` 替换为分配给你的服务端点。

2. 连接方式

服务支持多种连接方式。所有方式均通过 TLS 加密连接传输 JSON 消息。

2.1 基于 HTTP/1.1 的 WebSocket

标准 WebSocket 协议，最适合浏览器和大多数客户端应用。

会话超时：连续 55 秒没有数据交换时，连接将被关闭。

关键要求：如果在连续 55 秒的空闲期内没有交换 ping/pong，服务器会强制关闭 WebSocket 连接。会话将立即终止，且无法恢复。客户端必须重新建立连接并启动新会话。55 秒超时后没有宽限期。

连接保活（推荐）：客户端应每 20 秒发送一次 WebSocket ping 帧，以保持 55 秒的时间窗口内（确保超时前至少进行 2 次 ping/pong 交换）。详情参见第 2.6 节：连接保活。

连接 URL 格式：

```
wss://{host}/?pid={playerId}
```

URL 参数：

参数	必填	说明
pid	否	玩家标识符（用于会话亲和性）
player	否	pid 的别名
playerid	否	pid 的别名

Python 连接示例

提供两种 Python 客户端示例。两者发送的 JSON 协议消息完全相同，唯一区别是 I/O 模型：

客户端	库	适用场景
异步	<code>websockets</code>	<code>asyncio</code> 应用、大量并发连接

客户端	库	适用场景
同步	websocket-client	简单脚本、同步代码库

注意：URL 参数 pid 是可选的，仅用于会话亲和性和日志记录；实际身份认证通过第一条消息中的令牌完成。

异步客户端：参见 doc/examples/websocket_async_client.py

```
pip install websockets
```

同步客户端：参见 doc/examples/websocket_sync_client.py

```
pip install websocket-client
```

快速使用示例

异步 (websockets)：

```
import asyncio, json, websockets

async def quick_example():
    async with websockets.connect("wss://api.example.com/?pid=player1") as ws:
        # 身份认证 (令牌是服务运营方提供的 UUID)
        await ws.send(json.dumps({"token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70"}))
        auth = json.loads(await ws.recv())
        print(f"Auth: {auth}") # 保存 auth["sessionId"] 以便重连

asyncio.run(quick_example())
```

同步 (websocket-client)：

```
import json
from websocket import create_connection

ws = create_connection("wss://api.example.com/?pid=player1", timeout=15)
ws.send(json.dumps({"token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70"}))
auth = json.loads(ws.recv())
print(f"Auth: {auth}") # 保存 auth["sessionId"] 以便重连
ws.close()
```

生产环境提示：在实际应用中，应实现接收循环来处理 error 消息和断开连接的情况。完整流程参见 doc/examples/full_game_session.py。

2.2 基于 HTTP/2 的 WebSocket (RFC 8441)

HTTP/2 WebSocket 使用扩展 CONNECT 方法 (RFC 8441)。它支持在 HTTP/2 上建立 WebSocket 连接，并提供完整的多路复用能力，允许在单个 TCP 连接上传输多个 WebSocket 流。

会话超时：连续 55 秒没有数据交换时，连接将被关闭。

连接保活：客户端应每 20 秒发送一次 WebSocket ping 帧以维持连接。详情参见第 2.6 节：连接保活。

HTTP/2 扩展 CONNECT 伪头字段：

```
:method: CONNECT
:protocol: websocket
:scheme: https
:authority: api.example.com
:path: /?pid=player123
sec-websocket-version: 13
sec-websocket-key: dGhlIHhnbXBsZSBub25jZQ==
```

Python 连接示例 (hyper-h2)

完整实现参见 [doc/examples/http2_websocket_client.py](#)。

```
pip install h2 hpack
```

快速使用（运行完整示例）：

```
python doc/examples/http2_websocket_client.py
```

也可以在代码中导入并使用：

```
import sys
sys.path.insert(0, "doc/examples")
from http2_websocket_client import HTTP2WebSocketClient
import json

client = HTTP2WebSocketClient("api.example.com", 443)
client.connect("player123")

# 身份认证（令牌是服务运营方提供的 UUID）
client.send_ws_frame(json.dumps({"token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70"}))
auth = client.recv_ws_frame()
print(f"Auth: {auth}")

client.close()
```

2.3 HTTP/2 CONNECT 隧道

通用 HTTP/2 CONNECT 用于创建双向隧道。随后客户端通过该隧道执行标准 WebSocket 握手。

会话超时：连续 55 秒没有数据交换时，隧道将被关闭。

关键要求：如果在连续 55 秒的空闲期内没有交换 ping/pong，服务器会强制关闭隧道。会话将立即终止，且无法恢复。客户端必须重新建立连接并启动新会话。55 秒超时后没有宽限期。

连接保活（推荐）：客户端应每 20 秒发送一次 WebSocket ping 帧，以保持 55 秒的时间窗口内（确保超时前至少进行 2 次 ping/pong 交换）。详情参见第 2.6 节：连接保活。

HTTP/2 CONNECT 伪头字段：

```
:method: CONNECT
:authority: api.example.com
```

服务器返回 2xx 响应后，连接将成为传输 WebSocket 帧的透明隧道。

Python 连接示例

完整实现参见 [doc/examples/http2_connect_tunnel_client.py](#)。

```
pip install h2 hpack
```

快速使用（运行完整示例）：

```
python doc/examples/http2_connect_tunnel_client.py
```

也可以在代码中导入并使用：

```
import sys
sys.path.insert(0, "doc/examples")
from http2_connect_tunnel_client import HTTP2TunnelWebSocketClient

client = HTTP2TunnelWebSocketClient("api.example.com", 443)
client.connect("player123")

# 身份认证（令牌是服务运营方提供的 UUID）
client.send({"token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70"})
auth = client.recv()
print(f"Auth: {auth}")

# 发送牌局事件并获取决策……
client.close()
```

2.4 HTTP 请求/响应 API

对于无法维持 WebSocket 长连接的客户端，服务提供无状态的 HTTP API。同时支持 HTTP/1.1 和 HTTP/2。

端点：POST <https://{host}/api/command>

特性：

- 无状态请求/响应模式
- 适用于任何 HTTP/1.1 或 HTTP/2 客户端
- 每个请求的超时时间为 30 秒
- 身份认证后的每个后续请求都必须携带会话 ID
- 请求之间的最大空闲时间为 295 秒，超过后会话将被清除

关键要求：首次认证请求返回 `sessionId` 后，每个后续 HTTP 请求/响应（R/R）请求都必须携带该值。没有有效的 `sessionId`，请求将失败。

请求头：

请求头	必填	说明
Content-Type	是	必须为 application/json
X-Session-Id	是	认证响应中返回的会话 ID

示例：

```
# 第一个请求：身份认证（返回 sessionId）
curl -X POST "https://api.example.com/api/command?pid=player123" \
  -H "Content-Type: application/json" \
  -d '{"token": "your-token-here"}'

# 响应：{"result": true, "sessionId": "abc123def456", ...}

# 第二个请求：必须在请求头中携带 sessionId
curl -X POST "https://api.example.com/api/command?pid=player123" \
  -H "Content-Type: application/json" \
  -H "X-Session-Id: abc123def456" \
  -d '{"structType": "gameEvents", ...}'

# 缺少 X-Session-Id 请求头时，请求将返回 400 错误
```

HTTP/1.1 与 HTTP/2 性能对比：

项目	HTTP/1.1	HTTP/2
连接开销	新建 TCP 连接或使用 Keep-Alive	单个连接，多路复用流
队头阻塞	有（使用 Keep-Alive 时）	无（多路复用）
头部压缩	无	HPACK（约减少 80-90%）
TLS 握手	每个连接一次	每个连接一次
典型延迟	每个请求 50-100 毫秒	每个请求 50-80 毫秒
建议	适用于大多数情况	更适合高频请求

限制：

- 延迟高于 WebSocket 长连接（每个请求约增加 50-100 毫秒开销）
- 需要严格管理会话 ID，以维持牌局状态的连续性
- 不推荐用于对延迟敏感的应用

Python HTTP 客户端示例

完整实现参见 [doc/examples/http_api_client.py](#)。

```
pip install httpx
```

快速使用（运行完整示例）：

```
python doc/examples/http_api_client.py
```

也可以在代码中导入并使用：

```
import sys
sys.path.insert(0, "doc/examples")
from http_api_client import GatewayHTTPClient

client = GatewayHTTPClient(
    base_url="https://api.example.com",
    player_id="player123",
    token="a0321e5f-4d84-4e32-bef3-48814dfe7e70"
)

# 身份认证
auth = client.authenticate()
print(f"Session: {client.session_id}")

# 发送牌局事件并获取决策……
client.close()
```

cURL 示例

```
# 身份认证
curl --http2 -X POST "https://api.example.com/api/command?pid=player123" \
  -H "Content-Type: application/json" \
  -H "X-Player-Id: player123" \
  -d '{"token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70"}'

# 发送牌局事件
curl --http2 -X POST "https://api.example.com/api/command?pid=player123" \
  -H "Content-Type: application/json" \
```

```
-H "X-Player-Id: player123" \
-H "X-Session-Id: abc123def456" \
-d '{
  "structType": "gameEvents",
  "game": {
    "gameId": "test_game_001",
    "pokerNetwork": "TEST",
    "gameType": "NL",
    "bigBlind": 100,
    "ante": 0,
    "currency": "USD",
    "gameDate": "1706789012345",
    "numPlayers": 2,
    "buttonSetToSeat": 1
  },
  "events": [
    {"eventType": "playerSeated", "seat": 1, "name": "Hero", "stack": 10000},
    {"eventType": "playerSeated", "seat": 2, "name": "Villain", "stack": 10000}
  ]
}'
```

请求决策

```
curl --http2 -X POST "https://api.example.com/api/command?pid=player123" \
-H "Content-Type: application/json" \
-H "X-Player-Id: player123" \
-H "X-Session-Id: abc123def456" \
-d '{
  "structType": "getAnswer",
  "gameId": "test_game_001",
  "potForAlpha": 150,
  "delay": 0
}'
```

错误响应（仅限 HTTP R/R）

HTTP R/R API 的任何请求都可能返回协议层错误。这些错误独立于各个请求自身的错误（例如认证失败）。

HTTP 状态码	场景	错误消息
400	缺少 token 和 sessionId	{"error": "missing token: include token in body, or provide valid sessionId"}
400	sessionId 无效	{"error": "invalid sessionId"}
502	会话恢复失败	{"error": "session restoration failed: ..."}
504	后端超时	{"error": "backend timeout"}
500	服务器过载	{"error": "server overloaded, too many sessions"}
502	后端连接失败	{"error": "backend connection failed: ..."}

注意：WebSocket 协议没有协议层错误。WebSocket 响应中的所有错误均为 JSON 载荷内的请求级错误。

2.5 协议对比

本节从多个关键维度比较所有连接协议，帮助选择合适的方式。

对比矩阵

评价标准	基于 HTTP/1.1 的 WS	基于 HTTP/2 的 WS (RFC 8441)	HTTP/2 CONNECT 隧道	HTTP R/R API
易用性	★★★★★	★★☆☆☆	★★☆☆☆	★★★★★
可靠性	★★★★☆	★★★★☆	★★☆☆☆	★★★★★
性能	★★★★★	★★★★★	★★★★☆	★★☆☆☆
工具支持	★★★★★	★★☆☆☆	★★☆☆☆	★★★★★

详细对比

易用性

协议	评分	说明
基于 HTTP/1.1 的 WS	★★★★★	标准 WebSocket，文档丰富、握手简单、实现直接
基于 HTTP/2 的 WS (RFC 8441)	★★☆☆☆	需要支持 HTTP/2 扩展 CONNECT，帧处理复杂，可用示例较少
HTTP/2 CONNECT 隧道	★★☆☆☆	分两阶段建立连接（隧道 + WS 握手），需手动编码帧并管理 HTTP/2 状态
HTTP R/R API	★★★★★	标准 HTTP 请求/响应模式，适用于任何 HTTP 客户端，无需处理 WebSocket 的复杂机制

可靠性

协议	评分	说明
基于 HTTP/1.1 的 WS	★★★★☆	成熟且经过充分测试；单个连接可能因网络问题中断
基于 HTTP/2 的 WS (RFC 8441)	★★★★☆	内置流多路复用和自动流量控制，但实实验证相对较少
HTTP/2 CONNECT 隧道	★★★☆☆	隧道增加了复杂性，故障可能更难诊断
HTTP R/R API	★★★★★	无状态，每个请求相互独立；可以自动重试；没有连接状态丢失问题

性能

协议	评分	说明
基于 HTTP/1.1 的 WS	★★★★★	长连接，握手后开销极小，消息延迟低
基于 HTTP/2 的 WS (RFC 8441)	★★★★★	与 HTTP/1.1 WS 相同，并在多流场景下具备多路复用优势
HTTP/2 CONNECT 隧道	★★★★☆	隧道封装带来少量开销，但建立后为长连接
HTTP R/R API	★★☆☆☆	每条消息都有请求开销（约 50-100 毫秒），适合常规牌局节奏

工具与技术支持

协议	评分	库与工具
基于 HTTP/1.1 的 WS	★★★★★	websockets、websocket-client、aiohttp、浏览器、Postman、wscat
基于 HTTP/2 的 WS (RFC 8441)	★★☆☆☆	hyper-h2 (手动实现)、部分浏览器支持、库支持有限
HTTP/2 CONNECT 隧道	★★★☆☆	hyper-h2、httpx (支持有限)
HTTP R/R API	★★★★★	httpx、aiohttp、requests、curl、任意 HTTP 客户端、所有编程语言

按使用场景推荐

使用场景	推荐协议	原因
浏览器客户端	基于 HTTP/1.1 的 WS	浏览器原生 WebSocket API，兼容性最佳
桌面游戏客户端	基于 HTTP/1.1 的 WS	简单可靠，库支持完善
移动应用	基于 HTTP/1.1 的 WS	使用长连接时能效最佳
无服务器计算/云函数	HTTP R/R API	无状态、执行时间短、无需管理连接
简单集成/脚本	HTTP R/R API	可使用 curl --http2，无需 WebSocket 库
企业代理环境	HTTP/2 CONNECT 隧道	可能穿透限制严格的防火墙
多桌客户端	基于 HTTP/2 的 WS (RFC 8441)	通过单个连接处理多场牌局

各协议的依赖

协议	Python 依赖	说明
基于 HTTP/1.1 的 WS	websockets 或 websocket-client	推荐用于大多数场景
基于 HTTP/2 的 WS (RFC 8441)	h2、hpack	高级用法，适用于多路复用场景
HTTP/2 CONNECT 隧道	h2、hpack	高级用法，适用于受限网络
HTTP R/R API	httpx 或 requests	支持 HTTP/1.1 或 HTTP/2

```
# 安装 HTTP/1.1 WS 所需依赖 (推荐)
pip install websockets

# 安装 HTTP R/R API 所需依赖 (支持 HTTP/1.1 或 HTTP/2)
pip install httpx

# 安装高级 HTTP/2 协议所需依赖
pip install h2 hpack
```

2.6 连接保活

WebSocket 连接的空闲超时时间为 55 秒。为了维持长连接，客户端可以发送 ping 帧。仅在需要持续维持连接时，才推荐这样做。

警告：会话终止不可逆

如果在连续 55 秒的空闲期内没有收到任何 WebSocket 帧（包括 ping/pong），服务器会强制关闭连接。会话将立即、

永久终止；没有宽限期，无法通过重连恢复，也无法恢复会话。

客户端必须：

1. 重新建立连接（新的 WebSocket 连接）
2. 使用令牌重新认证
3. 启动新会话（原 sessionId 已失效）

55 秒超时后，无法以任何方式恢复会话。

为什么需要连接保活？

- 防止会话终止：如果不每 20 秒发送 ping，会话将在 55 秒后永久丢失
- 检测断开连接：通过 ping/pong 交换确认连接仍然有效
- 维持 NAT/防火墙状态：保活流量可防止 NAT 映射过期

实现指南

Ping 间隔（推荐）：每 20 秒发送一次 WebSocket ping 帧。这可确保在 55 秒的超时窗口内至少进行 2 次 ping/pong 交换，留出安全余量。

Python 示例（websockets 库）：

```
import asyncio
import websockets
import json

async def keepalive_loop(ws, interval=20):
    """定期发送 ping 帧。"""
    while True:
        await asyncio.sleep(interval)
        if ws.open:
            await ws.ping()
            print("Ping sent")

async def main():
    uri = "wss://api.example.com/?pid=player123"
    async with websockets.connect(uri) as ws:
        # 启动连接保活任务
        ping_task = asyncio.create_task(keepalive_loop(ws, interval=20))

        # 在此编写牌局逻辑……

        await ping_task

asyncio.run(main())
```

Python 示例（websocket-client 库）：

```

import websocket
import threading
import time

def ping_thread(ws, interval=20):
    """在后台线程中发送 ping 帧。"""
    while True:
        time.sleep(interval)
        if ws.sock and ws.sock.connected:
            ws.ping()
            print("Ping sent")

ws = websocket.create_connection("wss://api.example.com/?pid=player123")

# 启动 ping 线程
ping = threading.Thread(target=ping_thread, args=(ws, 20), daemon=True)
ping.start()

# 在此编写牌局逻辑……

ws.close()

```

JavaScript 示例：

```

const ws = new WebSocket('wss://api.example.com/?pid=player123');

const pingInterval = setInterval(() => {
    if (ws.readyState === WebSocket.OPEN) {
        ws.ping();
        console.log('Ping sent');
    }
}, 20000); // 20 秒

ws.onclose = () => clearInterval(pingInterval);
ws.onerror = () => clearInterval(pingInterval);

```

浏览器 WebSocket API：

```

const ws = new WebSocket('wss://api.example.com/?pid=player123');

ws.onopen = () => {
    // 启动 ping 定时器
    const pingInterval = setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) {
            ws.send(''); // 空帧充当 ping
            console.log('Ping sent');
        }
    }, 20000); // 20 秒

    ws.onclose = () => clearInterval(pingInterval);
};

```

汇总

参数	值	说明
Ping 间隔	20 秒	推荐用于连接保活，仅在需要持续连接时使用
连接超时	55 秒	如果未发送 ping，空闲 55 秒后会话将丢失
Pong 响应	自动	大多数库会自动处理 pong

注意：如果应用只需要定期连接（例如仅在进行牌局时连接），则无需发送 ping 帧。需要时重新连接即可，系统会创建新会话。

- 重新建立连接
- 使用令牌重新认证
- 获取新的 sessionId（旧 sessionId 已失效）

超时后无法恢复之前的会话。

3. 身份认证

连接建立后的第一条消息必须用于身份认证。

认证请求

```
{
  "token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70",
  "sessionId": "abc123def456",
  "descr": "Client v1.0"
}
```

字段	类型	必填	说明
token	字符串	是	认证令牌（UUID 格式）
sessionId	字符串	否	用于重连的先前会话 ID
descr	字符串	否	用于日志记录/调试的客户端描述

认证响应

成功：

```
{
  "result": true,
  "info": "Success",
  "sessionId": "a1b2c3d4e5f6..."
}
```

失败：

```
{
  "result": false,
  "info": "Error description"
}
```

错误消息

响应	含义
<code>{'result': false, 'info': 'No token data'}</code>	缺少令牌字段
<code>{'result': false, 'info': 'Incorrect token'}</code>	无法识别令牌
<code>{'result': false, 'info': 'Connections limit reached'}</code>	此令牌的活动连接数过多
<code>{'result': false, 'info': 'Daily limit number of requests has been reached'}</code>	已超出每日请求配额
<code>{'result': false, 'info': 'Token expired'}</code>	令牌已停用

会话恢复

会话机制允许客户端在断开连接后恢复，且不丢失牌局状态。

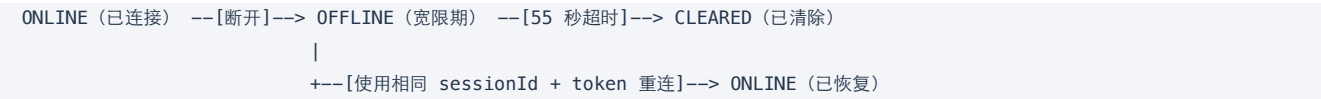
WebSocket 会话恢复

对于 WebSocket 连接，会话恢复方式如下：

要点：

- WebSocket 意外断开时，会话进入 OFFLINE（离线）状态
- 客户端必须在 55 秒内重新连接，并使用相同的 sessionId 重新认证
- 必须在不关闭之前连接的情况下尝试重连（假设是客户端网络问题）
- 空闲 55 秒后，会话将永久终止

会话状态机（WebSocket）：



约束：

约束	值	说明
宽限期	55 秒	会话清除前允许重连的时间窗口（WebSocket）
并发连接	每个会话 1 个	多个连接不能使用相同的 sessionId
保留的牌局	所有进行中的牌局	重连时恢复所有进行中的牌局

重连流程：

1. 客户端使用相同的 pid 重新连接 WebSocket 端点
2. 客户端发送认证消息，携带断开前的相同 token 和相同 sessionId
3. 服务查找 sessionId + token 匹配的 OFFLINE 会话
4. 如果在 55 秒内找到，牌局状态将转移到新连接
5. 客户端从中断处继续
6. 如果 55 秒内未找到，会话将被清除，客户端获得新的 sessionId

重要： 对于 WebSocket，必须在 55 秒超时之前重连。超时后 sessionId 将失效，无法恢复。

HTTP R/R 会话恢复

对于 HTTP R/R API，会话恢复更简单：

- 通过每个请求中的 X-Session-Id 请求头识别会话
- 如果 295 秒内没有任何请求，会话将被清除
- 只需携带 X-Session-Id 请求头，即可继续使用同一会话
- 无需重连，因为这是无状态 HTTP

约束：

约束	值	说明
会话超时	295 秒	两次请求之间、会话被清除前的时间窗口（HTTP R/R）
并发请求	每个会话 1 个	多个并发请求不能使用相同的 sessionId

重连流程：

1. 服务查找具有匹配会话的 OFFLINE 客户端
2. 如果在宽限期内找到，牌局状态将转移到新连接
3. 客户端可以从中断处继续
4. 如果未找到会话或会话已过期，则分配新的 sessionId

重要： sessionId 与认证令牌绑定，不能使用不同令牌恢复会话。

```
# 重连示例
async def reconnect(previous_session_id: str):
    ws = await websockets.connect("wss://api.example.com/?pid=player123")

    # 携带之前的会话 ID，并使用相同令牌
    await ws.send(json.dumps({
        "token": "a0321e5f-4d84-4e32-bef3-48814dfe7e70",
        "sessionId": previous_session_id
    }))

    response = await ws.recv()
    data = json.loads(response)

    if data.get("result"):
        # 会话已恢复，牌局已保留
        print(f"Reconnected with session: {data['sessionId']}")
    else:
        # 会话已过期或未找到，已创建新会话
        print(f"New session: {data.get('sessionId')}")
```

4. JSON 消息协议

所有消息都是采用 UTF-8 编码的 JSON 对象。structType 字段用于确定消息类型。

4.1 消息类型概览

structType	方向	用途	说明
gameEvents	客户端 → 服务端	发送当前牌局状态	
getAnswer	客户端 → 服务端	请求决策	
fullGameLog	客户端 → 服务端	发送完整牌局历史	
ping	客户端 → 服务端	会话保活（仅限 HTTP R/R）	参见第 2.4 节
playerAction	服务端 → 客户端	建议的行动	
(无)	服务端 → 客户端	认证响应	
(无)	服务端 → 客户端	错误响应（包含 error 字段）	

注意：

- 错误响应没有 structType 字段，应检查响应中是否存在 error 字段。
- ping 类型的 structType 仅适用于 HTTP R/R API，WebSocket 连接不支持。

4.2 gameEvents

向服务发送当前牌局状态和事件。

```
{
  "structType": "gameEvents",
  "game": {
    "gameId": "unique_game_identifier",
    "pokerNetwork": "WE",
    "gameType": "NL",
    "bigBlind": 200,
    "ante": 0,
    "currency": "USD",
    "strategyNumber": 1,
    "gameDate": "1706789012345",
    "numPlayers": 6,
    "buttonSetToSeat": 3
  },
  "events": [
    {"eventType": "playerSeated", "seat": 1, "name": "player1", "stack": 10000},
    {"eventType": "blindPosted", "name": "player1", "blindType": "SB", "amount": 100},
    {"eventType": "stageStarted", "stage": "preflop", "cards": ""},
    {"eventType": "handDealt", "name": "player1", "cards": "As,Kd"}
  ]
}
```

game 对象字段

字段	类型	必填	说明
gameId	字符串	是	唯一牌局标识符
pokerNetwork	字符串	是	扑克网络/房间代码（例如 "WE"、"GG"、"PS"）
gameType	字符串	是	游戏类型代码（参见 游戏类型 ）
bigBlind	整数	是	大盲注金额，单位为筹码
ante	整数	是	前注金额，单位为筹码（无前注时为 0）
currency	字符串	是	货币代码（例如 "USD"、"EUR"）
gameDate	字符串	是	牌局开始时间戳，以 Unix 纪元起算的毫秒数表示（例如 "1706789012345"）
numPlayers	整数	是	牌桌上的玩家人数
buttonSetToSeat	整数	是	庄家按钮所在的座位号
strategyNumber	整数	否	策略标识符
bombAnte	整数	否	Bomb 前注金额（用于 BOMB 变体）
squidMode	字符串	否	Squid 游戏模式（用于 SQUID 变体）
squidCost	整数	否	每个 squid 的价值，单位为筹码
squidNumber	整数	否	squid 总数
squidPlayed	整数	否	已完成争夺的 squid 数量
ripperPlayer	字符串	否	Ripper 玩家名称（用于 RIPPER 变体）
ripperBlind	整数	否	Ripper 盲注金额
ripperPot	整数	否	Ripper 底池金额
clubId	整数	否	俱乐部 ID。不同俱乐部不得使用相同 ID
clubName	字符串	否	俱乐部名称（规则与 clubId 相同）
clubRate	浮点数	否	相对于发送给本服务的货币的兑换比率。如果牌局使用筹码而非真实货币下注，此参数指定筹码与真实货币之间的兑换比率。例如，盲注为 10/20 筹码且每个筹码价值 0.1 美元时，应将此参数设为 0.1

响应行为

通过 WebSocket 发送的 `gameEvents` 是发送后无需等待响应的消息：

- 服务不发送确认响应
- 客户端不应等待响应
- 错误（如果有）将异步发送

对于 HTTP R/R API，`gameEvents` 返回一个空确认响应：

```
{"result": true}
```

4.3 getAnswer

针对当前牌局状态请求决策。

```
{
  "structType": "getAnswer",
  "gameId": "unique_game_identifier",
  "potForAlpha": 1500,
  "delay": 0
}
```

字段	类型	必填	说明
gameId	字符串	是	牌局标识符（必须与进行中的牌局匹配）
potForAlpha	整数	是	当前底池大小，单位为筹码（见下文）
delay	整数	否	最大响应延迟，单位为毫秒（默认值：9000）

potForAlpha 的含义：

potForAlpha 表示请求决策时的底池总额：

- 包括所有下注轮的全部下注
- 包括盲注和前注
- 不包括己方玩家仍需投入以跟注的筹码

示例：己方玩家位于小盲位（50），大盲注为 100。对手加注 300。底池 = 50 + 100 + 300 = 450。

请求己方玩家的决策时："potForAlpha": 450

响应：

服务返回一个 `playerAction` 对象（参见第 4.5 节）：

```
{
  "structType": "playerAction",
  "gameId": "unique_game_identifier",
  "name": "player1",
  "action": "raise",
  "amount": 600
}
```

4.4 fullGameLog

牌局结束后发送完整牌局历史，用于日志记录和分析。

```
{
  "structType": "fullGameLog",
  "game": {
    "gameId": "unique_game_identififier",
    "pokerNetwork": "WE",
    "gameType": "NL",
    "bigBlind": 200,
    "ante": 0,
    "currency": "USD",
    "gameDate": "1706789012345",
    "numPlayers": 6,
    "buttonSetToSeat": 3
  },
  "events": [
    {"eventType": "playerSeated", "seat": 1, "name": "player1", "stack": 10000},
    {"eventType": "playerSeated", "seat": 2, "name": "player2", "stack": 12000},
    {"eventType": "blindPosted", "name": "player1", "blindType": "SB", "amount": 100},
    {"eventType": "blindPosted", "name": "player2", "blindType": "BB", "amount": 200},
    {"eventType": "stageStarted", "stage": "preflop", "cards": ""},
    {"eventType": "handDealt", "name": "player1", "cards": "As,Kd"},
    {"eventType": "playerActed", "name": "player1", "action": "raise", "amount": 600},
    {"eventType": "playerActed", "name": "player2", "action": "fold", "amount": 0},
    {"eventType": "playerWon", "name": "player1", "amount": 800},
    {"eventType": "handShown", "name": "player1", "cards": "As,Kd"},
    {"eventType": "noHandShown", "name": "player2"},
    {"eventType": "gameOver"}
  ]
}
```

响应行为

`fullGameLog` 是发送后无需等待响应的消息：

- 服务不发送确认响应
- 用于牌局结束后的日志记录/分析
- 处理后，该牌局将从活动内存中移除

对于 HTTP R/R API，`fullGameLog` 返回一个空确认响应：

```
{"result": true}
```

4.5 playerAction（响应）

响应 `getAnswer` 返回的行动建议。

```
{
  "structType": "playerAction",
  "gameId": "unique_game_identififier",
  "name": "player1",
  "action": "raise",
  "amount": 600
}
```

字段	类型	说明
<code>structType</code>	字符串	始终为 "playerAction"
<code>gameId</code>	字符串	原始牌局标识符

字段	类型	说明
name	字符串	本次计算行动建议所针对的玩家名称
action	字符串	建议的行动：fold、check、call、bet、raise、all-in
amount	整数	下注/加注金额，单位为筹码（弃牌/过牌时为 0）

4.6 错误响应

错误响应没有 structType 字段，而是包含 error 和 gameId 字段：

```
{
  "error": "Description of the error",
  "gameId": "unique_game_identifier"
}
```

检测错误：

解析响应时，应先检查 error 字段：

```
response = json.loads(await ws.recv())
if "error" in response:
    print(f"Error: {response['error']}")
    # 处理错误
elif response.get("structType") == "playerAction":
    # 处理决策
elif "result" in response:
    # 处理认证响应
```

常见错误消息：

错误	原因
"Game not found"	gameId 与任何进行中的牌局均不匹配
"Invalid game state"	事件不一致或格式错误
"No token data"	认证消息中缺少令牌
"Incorrect token"	无法识别令牌
"Connections limit reached"	此令牌的活动连接数过多
"Daily limit reached"	已超出每日请求配额

连接终止：

发生超时或致命错误时，服务可能关闭 WebSocket 连接。客户端应处理以下情况：

- WebSocket 关闭帧（状态码 1000-1015）
- 意外断开连接（网络错误）

5. 牌局事件类型

事件用于描述扑克游戏中的行动和状态变化。

核心事件

playerSeated

玩家在牌桌入座。

```
{"eventType": "playerSeated", "seat": 1, "name": "player1", "stack": 10000}
```

字段	类型	说明
seat	整数	座位号（从 1 开始）
name	字符串	玩家名称/标识符
stack	整数	初始筹码量

blindPosted

玩家支付盲注。

```
{"eventType": "blindPosted", "name": "player1", "blindType": "SB", "amount": 100}
```

字段	类型	说明
name	字符串	玩家名称
blindType	字符串	SB、BB、ANTE、STRADDLE、POST、RIPPER
amount	整数	盲注金额，单位为筹码

stageStarted

新的下注轮开始。

```
{"eventType": "stageStarted", "stage": "flop", "cards": "7d,6d,5s"}
```

字段	类型	说明
stage	字符串	preflop、flop、turn、river、showdown
cards	字符串	公共牌（以逗号分隔，翻牌前为空）

handDealt

玩家收到底牌。

```
{"eventType": "handDealt", "name": "player1", "cards": "As,Kd"}
```

字段	类型	说明
name	字符串	玩家名称
cards	字符串	底牌（以逗号分隔）

playerActed

玩家执行行动。

```
{ "eventType": "playerActed", "name": "player1", "action": "raise", "amount": 500 }
```

字段	类型	说明
name	字符串	玩家名称
action	字符串	fold、check、call、bet、raise、all-in
amount	整数	此次行动的下注总额，单位为筹码（见下文）

amount 的含义（关键）：

amount 字段表示此次行动投入的筹码总额，而非追加的增量：

行动	amount 的含义	示例
fold	始终为 0	"amount": 0
check	始终为 0	"amount": 0
call	跟上当前下注所需的筹码总额	BB=100，跟注 → "amount": 100
bet	下注总额	首次下注 300 → "amount": 300
raise	加注幅度总额（raise-by）	加注 500 → "amount": 500
all-in	玩家的全部筹码	剩余筹码 1500，全下 → "amount": 1500

示例序列（BB=100）：

```
小盲支付 50：    { "blindType": "SB", "amount": 50 }
大盲支付 100：   { "blindType": "BB", "amount": 100 }
玩家加注：      { "action": "raise", "amount": 300 } // 加注幅度为 300，而非加注到 300
对手跟注：      { "action": "call", "amount": 300 }  // 跟上这 300
```

knownPlayerCards

已知的同伴底牌，用于提升牌局分析时的计算质量。如果你的两名或更多玩家位于同一牌桌，可以使用此事件交换同伴底牌信息。

```
{ "eventType": "knownPlayerCards", "name": "player1", "cards": "As,Kd" }
```

字段	类型	说明
name	字符串	同伴玩家名称
cards	字符串	同伴玩家的底牌

牌局结束事件（仅限 fullGameLog）

noHandShown

玩家盖牌，不展示底牌。

```
{ "eventType": "noHandShown", "name": "player2" }
```

handShown

玩家在摊牌时亮出底牌。

```
{"eventType": "handShown", "name": "player1", "cards": "As,Kd"}
```

playerWon

玩家赢得底池。

```
{"eventType": "playerWon", "name": "player1", "amount": 4500}
```

gameOver

牌局已结束。

```
{"eventType": "gameOver"}
```

6. 游戏类型及变体

6.1 标准游戏

代码	说明
NL	无限注德州扑克
PLO	底池限注奥马哈（4 张底牌）
PLO5C	底池限注奥马哈（5 张底牌）
PLO6C	底池限注奥马哈（6 张底牌）
NLP	无限注全下或弃牌玩法（Push-Fold）
OHPF	明牌全下或弃牌玩法（Open-face Push-Fold）
POFC	菠萝明牌十三张（Pineapple Open-Face Chinese）

6.2 BOMB 变体

Bomb 牌局采用特殊的前注结构，所有玩家支付相同的前注。

游戏类型代码：[NLB](#)、[PLOB](#)、[PLO5CB](#)

game 的附加字段：

```
{
  "gameType": "NLB",
  "bombAnte": 60
}
```

字段	类型	说明
bombAnte	整数	每位玩家的 Bomb 前注金额

示例：

```
{
  "structType": "gameEvents",
  "game": {
    "gameId": "bomb_game_001",
    "pokerNetwork": "PN",
    "gameType": "NLB",
    "bigBlind": 20,
    "ante": 0,
    "bombAnte": 60,
    "currency": "CHM",
    "gameDate": "1750064327000",
    "numPlayers": 4,
    "buttonSetToSeat": 3
  },
  "events": [
    {"eventType": "playerSeated", "seat": 2, "name": "Player1", "stack": 1636},
    {"eventType": "playerSeated", "seat": 3, "name": "Player2", "stack": 3027},
    {"eventType": "playerSeated", "seat": 4, "name": "Player3", "stack": 2663},
    {"eventType": "playerSeated", "seat": 5, "name": "Player4", "stack": 800},
    {"eventType": "blindPosted", "name": "Player3", "blindType": "ANTE", "amount": 60},
    {"eventType": "blindPosted", "name": "Player4", "blindType": "ANTE", "amount": 60},
    {"eventType": "blindPosted", "name": "Player1", "blindType": "ANTE", "amount": 60},
    {"eventType": "blindPosted", "name": "Player2", "blindType": "ANTE", "amount": 60},
    {"eventType": "stageStarted", "stage": "preflop"},
    {"eventType": "handDealt", "name": "Player3", "cards": "2c,5c"},
    {"eventType": "stageStarted", "stage": "flop", "cards": "Kd,Qd,5s"}
  ]
}
```

6.3 DOUBLE BOARD BOMB 变体

Bomb 牌局采用特殊的前注结构，所有玩家支付相同的前注。

游戏类型代码：NLDBB

game 的附加字段：

```
{
  "gameType": "NLDBB",
  "bombAnte": 60
}
```

字段	类型	说明
bombAnte	整数	每位玩家的 Bomb 前注金额

附加事件类型：

```
{"eventType": "stageStarted", "stage": "flop", "cards": "2c,Ks,6h", "doubleBoardCards": "Ts,5c,8d"}
```

示例：

```
{
  "structType": "fullGameLog",
  "game": {
    "gameId": "nlddb_game_001",
    "pokerNetwork": "PN",
    "gameType": "NLDBB",
    "bigBlind": 50,
    "ante": 0,
```

```

    "currency": "USD",
    "gameDate": "1778727848673",
    "numPlayers": 5,
    "buttonSetToSeat": 5,
    "strategyNumber": 0,
    "bombAnte": 150
  },
  "events": [
    {"eventType": "playerSeated", "seat": 1, "name": "Player1", "stack": 2588},
    {"eventType": "playerSeated", "seat": 2, "name": "Player2", "stack": 4961},
    {"eventType": "playerSeated", "seat": 3, "name": "Player3", "stack": 2985},
    {"eventType": "playerSeated", "seat": 5, "name": "Player4", "stack": 4500},
    {"eventType": "playerSeated", "seat": 6, "name": "Player5", "stack": 3280},
    {"eventType": "blindPosted", "name": "Player1", "blindType": "ANTE", "amount": 150},
    {"eventType": "blindPosted", "name": "Player2", "blindType": "ANTE", "amount": 150},
    {"eventType": "blindPosted", "name": "Player3", "blindType": "ANTE", "amount": 150},
    {"eventType": "blindPosted", "name": "Player4", "blindType": "ANTE", "amount": 150},
    {"eventType": "blindPosted", "name": "Player5", "blindType": "ANTE", "amount": 150},
    {"eventType": "stageStarted", "stage": "preflop", "cards": ""},
    {"eventType": "handDealt", "name": "Player4", "cards": "5d,Qd"},
    {"eventType": "stageStarted", "stage": "flop", "cards": "2c,Ks,6h", "doubleBoardCards": "Ts,5c,8d"},
    {"eventType": "playerActed", "name": "Player5", "action": "check", "amount": 0},
    {"eventType": "playerActed", "name": "Player1", "action": "raise", "amount": 375},
    {"eventType": "playerActed", "name": "Player2", "action": "call", "amount": 375},
    {"eventType": "playerActed", "name": "Player3", "action": "fold", "amount": 0},
    {"eventType": "playerActed", "name": "Player4", "action": "fold", "amount": 0},
    {"eventType": "playerActed", "name": "Player5", "action": "fold", "amount": 0},
    {"eventType": "stageStarted", "stage": "turn", "cards": "Jd", "doubleBoardCards": "Jc"},
    {"eventType": "playerActed", "name": "Player1", "action": "raise", "amount": 750},
    {"eventType": "playerActed", "name": "Player2", "action": "call", "amount": 750},
    {"eventType": "stageStarted", "stage": "river", "cards": "Qs", "doubleBoardCards": "Th"},
    {"eventType": "playerActed", "name": "Player1", "action": "all-in", "amount": 1313},
    {"eventType": "playerActed", "name": "Player2", "action": "fold", "amount": 0},
    {"eventType": "noHandShown", "name": "Player1"},
    {"eventType": "playerWon", "name": "Player1", "amount": 2735},
    {"eventType": "gameOver"}
  ]
}

```

6.4 SQUID 变体

Squid 牌局包含一种特殊的小游戏，玩家在其中争夺“squid（鱿鱼）”。

游戏类型代码：NLSQ

game 的附加字段：

```

{
  "gameType": "NLSQ",
  "squidMode": "HUNT",
  "squidCost": 300,
  "squidNumber": 5,
  "squidPlayed": 2
}

```

字段	类型	说明
squidMode	字符串	UNDEFINED、STAND_UP（每位玩家最多 1 个）、HUNT（不限数量）
squidCost	整数	一个 squid 的价值，单位为筹码

字段	类型	说明
squidNumber	整数	牌局中的 squid 总数
squidPlayed	整数	已赢得的 squid 数量

附加事件类型：

```
{
  "eventType": "playerHasSquid", "name": "player2", "count": 1}
{"eventType": "playerGotSquid", "name": "player3"}
{"eventType": "squidPenalty", "name": "player1", "amount": 900}
{"eventType": "squidPayment", "name": "player3", "amount": 300}
```

事件	说明
playerHasSquid	玩家当前的 squid 数量
playerGotSquid	玩家赢得一个 squid（仅限 fullGameLog）
squidPenalty	玩家支付 squid 罚金（仅限 fullGameLog）
squidPayment	玩家收到 squid 奖金（仅限 fullGameLog）

示例：

```
{
  "structType": "fullGameLog",
  "game": {
    "gameId": "squid_game_001",
    "pokerNetwork": "PN",
    "gameType": "NLSQ",
    "bigBlind": 1000,
    "ante": 0,
    "currency": "USD",
    "gameDate": "1749551962000",
    "numPlayers": 3,
    "buttonSetToSeat": 5,
    "squidMode": "HUNT",
    "squidCost": 300,
    "squidNumber": 5,
    "squidPlayed": 2
  },
  "events": [
    {"eventType": "playerSeated", "seat": 2, "name": "player1", "stack": 20000},
    {"eventType": "playerSeated", "seat": 5, "name": "player2", "stack": 20000},
    {"eventType": "playerSeated", "seat": 6, "name": "player3", "stack": 20000},
    {"eventType": "playerHasSquid", "name": "player2", "count": 2},
    {"eventType": "blindPosted", "name": "player3", "blindType": "SB", "amount": 500},
    {"eventType": "blindPosted", "name": "player1", "blindType": "BB", "amount": 1000},
    {"eventType": "stageStarted", "stage": "preflop"},
    {"eventType": "handDealt", "name": "player1", "cards": "7s,As"},
    {"eventType": "playerActed", "name": "player2", "action": "fold", "amount": 0},
    {"eventType": "playerActed", "name": "player3", "action": "call", "amount": 500},
    {"eventType": "playerActed", "name": "player1", "action": "check", "amount": 0},
    {"eventType": "stageStarted", "stage": "flop", "cards": "8c,Td,Jc"},
    {"eventType": "playerActed", "name": "player3", "action": "raise", "amount": 4000},
    {"eventType": "playerActed", "name": "player1", "action": "fold", "amount": 0},
    {"eventType": "playerWon", "name": "player3", "amount": 6000},
    {"eventType": "playerGotSquid", "name": "player3"},
    {"eventType": "squidPenalty", "name": "player1", "amount": 900},
    {"eventType": "squidPayment", "name": "player2", "amount": 600},
```

```
    {"eventType": "squidPayment", "name": "player3", "amount": 300},
    {"eventType": "gameOver"}
  ]
}
```

6.5 RIPPER 变体

Ripper 牌局中有一名特殊的 “ripper” 玩家，需要支付额外盲注，并可赢得一个边池。

游戏类型代码：包含 RIP（例如 NLRIP）

game 的附加字段：

```
{
  "gameType": "NLRIP",
  "ripperPlayer": "player3",
  "ripperBlind": 200,
  "ripperPot": 500
}
```

字段	类型	说明
ripperPlayer	字符串	ripper 玩家名称
ripperBlind	整数	Ripper 盲注金额
ripperPot	整数	当前 ripper 底池金额

附加事件类型：

```
{"eventType": "blindPosted", "name": "player3", "blindType": "RIPPER", "amount": 200}
{"eventType": "playerIsRipper", "name": "player3"}
{"eventType": "playerWonRipperPot", "name": "player3", "amount": 500}
```

6.6 POFC 变体

游戏类型代码：POFC

game 的附加字段：

```
{
  "gameType": "POFC",
  "gameSubType": 0,
  "kush": 100
}
```

字段	类型	说明
gameSubType	整数	游戏变体
kush	整数	每一分对应的筹码价值

gameSubType 值	牌组大小	说明
0	52 张牌	标准菠萝明牌十三张
1	52 张牌	渐进式菠萝明牌十三张
2	54 张牌（鬼牌 BJ、RJ）	渐进式菠萝明牌十三张
3	54 张牌（鬼牌 BJ、RJ）	终极菠萝明牌十三张

gameSubType 值	牌组大小	说明
4	52 张牌	终极菠萝明牌十三张

附加事件类型：

```
{
  "eventType": "playerFantasy", "name": "pid0", "type": "WITHOUT_FANTASY"}
{"eventType": "playerActed", "name": "pid0", "front": "Ks", "middle": "3h,Th", "back": "Ad,Ah", "discard": ""}
{"eventType": "handShown", "name": "pid0", "front": "Ks,Qs,8d", "middle": "3h,Th,Kh,2h,4h", "back": "Ad,Ah,9c,9s,9d"}
}
```

梦幻模式类型	说明
WITHOUT_FANTASY	标准玩法。玩家在初始阶段收到 5 张牌，随后 4 个阶段中每个阶段收到 3 张牌。
UNDEFINED_FANTASY	玩家处于梦幻模式（Fantasyland），进入原因未知。通常收到 14 张牌，并在常规游戏阶段之外完成决策。
QQ_FANTASY	玩家上一局在头道组成一对 Q，因此进入梦幻模式。
KK_FANTASY	玩家上一局在头道组成一对 K，因此进入梦幻模式。
AA_FANTASY	玩家上一局在头道组成一对 A，因此进入梦幻模式。
TREE_OF_KIND_FANTASY	玩家上一手牌在头道组成三条（原文为“Tree of Kind”），因此进入梦幻模式。

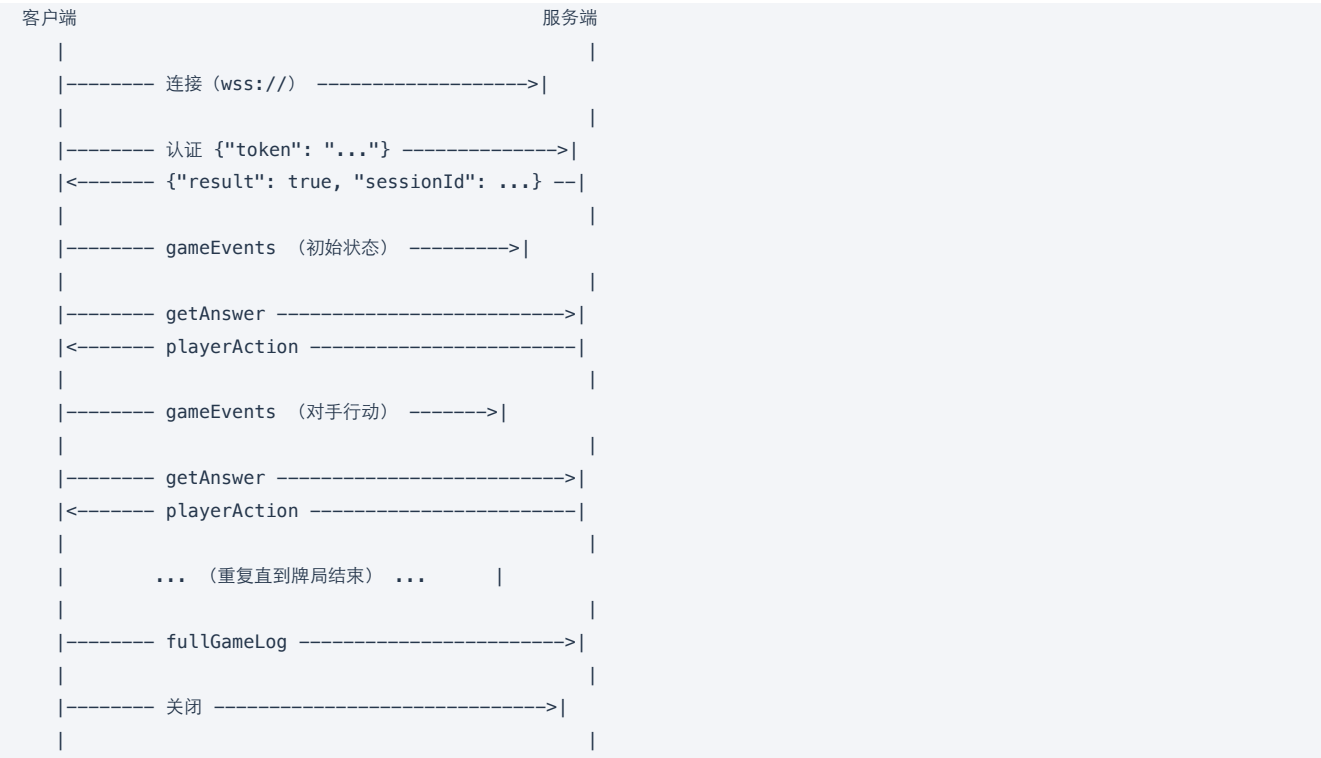
示例：

```
{
  "structType": "fullGameLog",
  "game": {
    "gameId": "0123456_pid0",
    "pokerNetwork": "PMTR",
    "gameType": "POFC",
    "gameSubType": 0,
    "gameDate": "1783586778622",
    "numPlayers": 2,
    "buttonSetToSeat": 2,
    "kush": 100,
    "currency": "USD"
  },
  "events": [
    {"eventType": "playerSeated", "name": "pid0", "seat": 1, "stack": 10000},
    {"eventType": "playerSeated", "name": "pid1", "seat": 2, "stack": 10000},
    {"eventType": "playerFantasy", "name": "pid0", "type": "WITHOUT_FANTASY"},
    {"eventType": "playerFantasy", "name": "pid1", "type": "WITHOUT_FANTASY"},
    {"eventType": "stageStarted", "stage": "street_1", "cards": ""},
    {"eventType": "handDealt", "name": "pid0", "cards": "3h,Ad,Ah,Ks,Th"},
    {"eventType": "playerActed", "name": "pid0", "front": "Ks", "middle": "3h,Th", "back": "Ad,Ah", "discard": ""},
    {"eventType": "playerActed", "name": "pid1", "front": "Kd", "middle": "7d,7s", "back": "2d,3d", "discard": ""},
    {"eventType": "stageStarted", "stage": "street_2", "cards": ""},
    {"eventType": "handDealt", "name": "pid0", "cards": "Js,Kh,Qs"},
    {"eventType": "playerActed", "name": "pid0", "front": "Qs", "middle": "Kh", "back": "", "discard": "Js"},
    {"eventType": "playerActed", "name": "pid1", "front": "Qc", "middle": "Ts", "back": "", "discard": ""},
    {"eventType": "stageStarted", "stage": "street_3", "cards": ""},
    {"eventType": "handDealt", "name": "pid0", "cards": "9c,9s,Jc"},
    {"eventType": "playerActed", "name": "pid0", "front": "", "middle": "", "back": "9c,9s", "discard": "Jc"},
    {"eventType": "playerActed", "name": "pid1", "front": "8h", "middle": "", "back": "4d", "discard": ""},
    {"eventType": "stageStarted", "stage": "street_4", "cards": ""},
    {"eventType": "handDealt", "name": "pid0", "cards": "2h,3s,8d"},
  ]
}
```

```
    {"eventType": "playerActed", "name": "pid0", "front": "8d", "middle": "2h", "back": "", "discard": "3s"},
    {"eventType": "playerActed", "name": "pid1", "front": "", "middle": "Ac", "back": "5d", "discard": ""},
    {"eventType": "stageStarted", "stage": "street_5", "cards": ""},
    {"eventType": "handDealt", "name": "pid0", "cards": "4h,8s,9d"},
    {"eventType": "playerActed", "name": "pid0", "front": "", "middle": "4h", "back": "9d", "discard": "8s"},
    {"eventType": "playerActed", "name": "pid1", "front": "", "middle": "As", "back": "Td", "discard": ""},
    {"eventType": "stageStarted", "stage": "showdown", "cards": ""},
    {"eventType": "handShown", "name": "pid0", "front": "Ks,Qs,8d", "middle": "3h,Th,Kh,2h,4h", "back": "Ad,Ah,9c,9s,9d"},
    {"eventType": "handShown", "name": "pid1", "front": "Kd,Qc,8h", "middle": "7d,7s,Ts,Ac,As", "back": "2d,3d,4d,5d,Td"},
    {"eventType": "playerWon", "name": "pid0", "amount": 12},
    {"eventType": "gameOver"}
  ]
}
```

7. 协议流程

标准牌局流程



HTTP 请求/响应流程


```
# 从首个请求获取 sessionId 后，在所有后续请求中使用它
curl -X POST "https://api.example.com/api/command?pid=player123" \
-H "Content-Type: application/json" \
-H "X-Session-Id: abc123def456" \
-d '{"structType": "gameEvents", ...}'
```

增量更新

重要： 每条 `gameEvents` 消息必须包含从本手牌开始的完整有序事件列表。服务会将其与此前收到的事件比较，以去除重复事件。

```
// 初始 gameEvents：事件 1-6
{
  "structType": "gameEvents",
  "game": { /* 完整 game 对象 */ },
  "events": [
    {"eventType": "playerSeated", ...}, // 事件 1
    {"eventType": "playerSeated", ...}, // 事件 2
    {"eventType": "blindPosted", ...}, // 事件 3
    {"eventType": "blindPosted", ...}, // 事件 4
    {"eventType": "stageStarted", ...}, // 事件 5
    {"eventType": "handDealt", ...} // 事件 6
  ]
}

// 后续 gameEvents：事件 1-7（所有已有事件 + 新事件）
{
  "structType": "gameEvents",
  "game": { /* 相同的 game 对象 */ },
  "events": [
    {"eventType": "playerSeated", ...}, // 事件 1（不变）
    {"eventType": "playerSeated", ...}, // 事件 2（不变）
    {"eventType": "blindPosted", ...}, // 事件 3（不变）
    {"eventType": "blindPosted", ...}, // 事件 4（不变）
    {"eventType": "stageStarted", ...}, // 事件 5（不变）
    {"eventType": "handDealt", ...}, // 事件 6（不变）
    {"eventType": "playerActed", ...} // 事件 7（新增）
  ]
}
```

为什么需要完整历史？ 服务通过比较事件列表的前缀来识别新增事件。这样可以：

- 简化客户端实现（只需追加新事件）
- 无需序列号即可可靠去重
- 从消息遗漏中恢复

不要只发送新增事件，服务需要完整的有序列表来保持一致性。

8. 超时与限制

连接超时

协议	超时类型	时长	说明
WebSocket	身份认证	15 秒	必须在连接后 15 秒内完成认证
WebSocket	会话空闲	55 秒	空闲 55 秒后关闭连接（可通过 ping/pong 防止）
WebSocket	会话恢复	55 秒	55 秒宽限期后清除 OFFLINE 客户端数据
HTTP R/R	会话空闲	295 秒	空闲 295 秒后清除会话（可通过 {"structType": "ping"} 防止）
HTTP R/R	请求超时	30 秒	每个请求的超时时间

注意：建议约每 20 秒发送一次 ping/pong，以保持 55 秒的时间窗口内。HTTP R/R 是无状态的，每个请求只需携带 X-Session-Id 请求头；若维持会话，则使用 {"structType": "ping"}。

消息限制

限制	值	说明
最小消息长度	30 个字符	少于 30 个字符的消息将被拒绝
令牌连接数限制	可配置	每个令牌的最大同时连接数
每日请求数限制	可配置	每个令牌每天的最大请求数（可选）

9. 完整示例

完整牌局会话（Python）

完整可运行示例参见 [doc/examples/full_game_session.py](#)，演示了：

1. WebSocket 连接和身份认证
2. 使用 `gameEvents` 发送初始牌局状态
3. 使用 `getAnswer` 请求决策
4. 处理牌局的增量更新
5. 在牌局结束时发送 `fullGameLog`

```
pip install websockets
python doc/examples/full_game_session.py
```

扑克牌记法

每张牌以两个字符的字符串表示：

- 点数：2、3、4、5、6、7、8、9、T、J、Q、K、A
- 花色：c（梅花）、d（方块）、h（红桃）、s（黑桃）

示例：

- As = 黑桃 A
- Kd = 方块 K

- Th = 红桃 10

- 2c = 梅花 2

多张牌用逗号分隔："As,Kd" 或 "7d,6d,5s"

附录 A：扑克牌表示法

点数	符号
二	2
三	3
四	4
五	5
六	6
七	7
八	8
九	9
十	T
J（杰克）	J
Q（王后）	Q
K（国王）	K
A（艾斯）	A

花色	符号
梅花	c
方块	d
红桃	h
黑桃	s

附录 B：行动类型

行动	说明
fold	弃牌，放弃争夺底池
check	过牌（没有需要跟注的下注时）
call	跟注，跟上当前下注
bet	下注，本轮首次投入下注
raise	加注，提高当前下注
all-in	全下，投入全部筹码

附录 C：盲注类型

类型	说明
SB	小盲注
BB	大盲注
ANTE	前注
STRADDLE	自愿盲注（2 倍大盲注）
POST	补交盲注（中途入局）
RIPPER	Ripper 盲注（RIPPER 变体）

附录 D：错误处理

```
async def handle_response(ws):
    response = await ws.recv()
    data = json.loads(response)

    if "error" in data:
        print(f"Error: {data['error']}")
        # 按需处理错误
        return None

    if data.get("structType") == "playerAction":
        return data

    if "result" in data: # 认证响应
        if not data["result"]:
            print(f"Auth failed: {data.get('info')}")
            return None
        return data

    return data
```